

BCC Binary-Junction Node Splitting in ParaDiS

File-by-File Code and Design Guide

ParaDiS-Extended Development Notes

August 2026

Abstract

This note documents the implementation of three-arm BCC binary-junction splitting in the serial and parallel ParaDiS topology paths. Unlike a high-level feature summary, it shows the code added or changed in every implementation and regression file and explains the purpose of each change. The port follows the ExaDiS/OpenDiS implementation pinned at commit 351607fa0f04aa34d29ee253438edde994473fd6, with ParaDiS-specific adaptations for node constraints, remeshing, and serialized cross-domain topology operations.

Contents

1	Scope, lineage, and reading convention	3
2	Public definitions and interfaces	3
2.1	include/Node.h	3
2.2	include/Param.h	3
2.3	include/Topology.h	4
2.4	include/OpList.h	4
3	Parameter and initialization plumbing	4
3.1	src/Initialize.cc	4
3.2	src/InputSanity.cc	5
3.3	src/ParadisInit.cc	5
3.4	src/Param.cc	5
4	Shared classifier and serial topology path	5
4.1	src/Topology.cc	5
5	Parallel candidate and force evaluation	11
5.1	src/SMNSupport.cc	11
5.2	src/SMNEvalSplitForces.cc	12
5.3	src/SMNEvalMultiNodeSplits.cc	13
6	Remesh preservation and MPI propagation	15
6.1	src/Remesh.cc	15
6.2	src/RemoteOps.cc	16
6.3	src/FixRemesh.cc	17

7	Regression fixture	17
7.1	tests/bcc_binary_junction_node.ctrl	17
7.2	tests/bcc_binary_junction_node.data	19
7.3	Commands and expected result	20
8	End-to-end meaning of the change	20

1 Scope, lineage, and reading convention

The functional implementation is the difference from the pre-feature base `efd4069bfb9540a2ed76ccbe9892ed2c4415` through `3cb6c8bd92d7407a5b80b595d116574dc6b86f7f`. The first implementation commit is `74f18516eb24587c65d123e77a9713c71f38d2f0`; the second completes MPI constraint propagation, remesh protection, and the regression fixture. The exact change set can be inspected with:

```
1 git diff efd4069bfb9540a2ed76ccbe9892ed2c441597f7 \
2          3cb6c8bd92d7407a5b80b595d116574dc6b86f7f -- \
3          include src tests
```

The pinned reference is available at:

https://github.com/llnl/exadis/tree/351607fa0f04aa34d29ee253438edde994473fd6/src/topology_types

Every changed file has a subsection below. Listings reproduce the final code for the introduced behavior, with only enough surrounding context to make each fragment understandable. Large functions are divided into focused fragments rather than copied wholesale. Therefore the listings are a design and change guide, while the repository source remains authoritative. Long source lines are occasionally reindented to fit the page without changing their meaning. Comments beginning with “Existing” are editorial elisions, not new comments in the C++ source.

The 17 changed implementation and test files fall into five groups:

- public definitions: four headers under `include`;
- parameter and initialization plumbing: four source files;
- serial and parallel topology evaluation: four source files;
- remesh and MPI propagation: three source files; and
- regression inputs: one control file and one nodal-data file.

2 Public definitions and interfaces

2.1 `include/Node.h`

New lines, with existing enumeration entries shown for context

```
1 #define CORNER_NODE          0x00000010    /* 16: Preserve a physical corner */
2                                     /*      during mesh coarsening.      */
```

Meaning. The new constraint bit marks the degree-two node created by a successful binary-junction split. It is a topology state, not a temporary flag: remesh uses it to distinguish a physical kink from an ordinary degree-two node that may be coarsened away. Decimal value 16 is deliberately disjoint from the existing pinning and surface bits.

2.2 `include/Param.h`

Code added

```
1 int split3node;                /* Toggle BCC binary-junction */
2                                     /* three-arm node splitting. */
```

Meaning. This member carries the runtime control-file switch through `Param_t`. It lets a user enable or disable degree-three BCC splitting without changing the established treatment of nodes with four or more arms.

2.3 include/Topology.h

Code added

```
1 int BCC_binary_junction_node(Home_t *home, Node_t *node,
2     real8 tjunc[3], int *planarJunc);
```

Meaning. The declaration exposes one shared classifier to the serial topology path and the parallel candidate/force stages. The return value is the junction-arm index or -1 . The output vector `tjunc` is the unit junction-line direction, and `planarJunc` reports whether the parent-plane geometry is planar and therefore excluded from this split mechanism.

2.4 include/OpList.h

Code added

```
1 typedef enum {
2     REM_OP_CHANGE_CONN,
3     REM_OP_INSERT_ARM,
4     REM_OP_REMOVE_NODE,
5     REM_OP_CHANGE_BURG,
6     REM_OP_SPLIT_NODE,
7     REM_OP_RESET_COORD,
8     REM_OP_RESET_SEG_FORCE1,
9     REM_OP_RESET_SEG_FORCE2,
10    REM_OP_MARK_FORCE_OBSOLETE,
11    REM_OP_RESET_PLANE,
12    REM_OP_RESET_CONSTRAINT
13 } OpType_t;
14
15 typedef struct {
16     int opType;
17     Tag_t tag1;
18     int constraint;
19 } RemOpResetConstraint_t;
20
21 void AddOpResetConstraint(Home_t *home, Tag_t *tag1, int constraint);
```

Meaning. ParaDiS communicates topology changes as packed operation records. The new record transmits a node tag plus the complete integer constraint bitmask. Its operation identifier is appended to the enumeration, preserving the numeric values of all existing operation types. All MPI ranks must still run the same rebuilt executable because the byte-stream protocol now includes this record.

3 Parameter and initialization plumbing

3.1 src/Initialize.cc

Code changed

```
1 allValidConstraints = PINNED_X | PINNED_Y | PINNED_Z |  
2                       SURFACE_NODE | CORNER_NODE |  
3                       MULTI_JUNCTION_NODE |  
4                       LOOP_NODE | CROSS_NODE;
```

Meaning. Restart and input validation now accepts constraint bit 16. Without this addition, a restart written after a binary-junction split would be rejected as containing an unknown node constraint.

3.2 src/InputSanity.cc

Code added

```
1 param->split3node = (param->split3node != 0);
```

Meaning. Any nonzero input is normalized to one. Downstream candidate logic can treat `split3node` as a strict Boolean rather than preserving arbitrary integer values from a control file.

3.3 src/ParadisInit.cc

Code added

```
1 printf("|   split BCC three-arm nodes   : %s\n",  
2        p->split3node ? "yes" : "no");
```

Meaning. The startup summary makes the effective state visible before stepping begins. This is especially useful when comparing otherwise identical serial-selector and parallel-selector regression runs.

3.4 src/Param.cc

Code added or changed

```
1 BindVar(CPList, "split3node", &param->split3node,  
2         V_INT, 1, VFLAG_NULL);  
3 param->split3node = 1;  
4  
5 printf("   splitMultiNodeAlpha           : %+lf\n",  
6        p->splitMultiNodeAlpha);  
7 printf("   split3node                     : %d\n\n",  
8        p->split3node);
```

Meaning. `CtrlParamInit()` registers the control keyword and enables it by default. `Print_Param()` includes the value in the detailed parameter dump. Existing control files therefore gain the feature automatically, while `split3node = 0` restores the former degree-four minimum.

4 Shared classifier and serial topology path

4.1 src/Topology.cc

This file contains the shared BCC classifier and the serial implementation in `SplitMultiNodes()`.

4.1.1 Removing stale corner state

Code added

```
1 if (HAS_ANY_OF_CONSTRAINTS(localNode->constraint, CORNER_NODE) &&
2   (localNode->numNbrs != 2)) {
3   REMOVE_CONSTRAINTS(localNode->constraint, CORNER_NODE);
4 }
5
6 if (HAS_ANY_OF_CONSTRAINTS(ghostNode->constraint, CORNER_NODE) &&
7   (ghostNode->numNbrs != 2)) {
8   REMOVE_CONSTRAINTS(ghostNode->constraint, CORNER_NODE);
9 }
```

Meaning. InitTopologyExemptions() enforces the invariant that a corner marker is meaningful only on a degree-two node. If a later collision or topology event changes the degree, native and ghost copies lose stale corner state during the next InitTopologyExemptions() pass.

4.1.2 Recognizing a BCC binary junction

Code added or moved, with both adjustment calls shown for context

```
1 int BCC_binary_junction_node(Home_t *home, Node_t *node,
2                             real8 tjunc[3], int *planarJunc)
3 {
4     int    armID;
5     int    binaryJunc;
6     int    numArmEdge, numArmGlide, numArmJunc;
7     real8  ndir[3][3];
8     real8  eps, angleTol;
9     Param_t *param;
10
11     binaryJunc = -1;
12     numArmEdge = 0;
13     numArmGlide = 0;
14     numArmJunc = 0;
15
16     VECTOR_ZERO(tjunc);
17     *planarJunc = 0;
18
19     if ((node == (Node_t *)NULL) || (node->numNbrs != 3)) {
20         return(-1);
21     }
22
23     param = home->param;
24     eps = 1.0e-12;
25     angleTol = 5.0 * M_PI / 180.0;
26
27     for (armID = 0; armID < node->numNbrs; armID++) {
28         real8  bMag, lineMag2;
29         real8  burg[3], lineDir[3], normal[3];
30         Node_t *nbrNode;
31
32         nbrNode = GetNeighborNode(home, node, armID);
33
34         if (nbrNode == (Node_t *)NULL) {
35             continue;
36         }
37
```

```

38     lineDir[X] = nbrNode->x - node->x;
39     lineDir[Y] = nbrNode->y - node->y;
40     lineDir[Z] = nbrNode->z - node->z;
41
42     ZImage(param, &lineDir[X], &lineDir[Y], &lineDir[Z]);
43     lineMag2 = DotProduct(lineDir, lineDir);
44
45     if (lineMag2 < eps) {
46         continue;
47     }
48
49     NormalizeVec(lineDir);
50
51     burg[X] = node->burgX[armID];
52     burg[Y] = node->burgY[armID];
53     burg[Z] = node->burgZ[armID];
54     bMag = Normal(burg);
55
56     if (bMag > (1.0 + eps)) {
57         VECTOR_COPY(tjunc, lineDir);
58         binaryJunc = armID;
59         numArmJunc++;
60     } else {
61         CrossVector(burg, lineDir, normal);
62
63         if (fabs(DotProduct(normal, normal)) > 1.0e-2) {
64             VECTOR_COPY(ndir[numArmEdge], normal);
65             numArmEdge++;
66         }
67
68         numArmGlide++;
69     }
70 }
71
72 if ((numArmJunc == 1) && (numArmGlide == 2)) {
73     if (numArmEdge == 2) {
74         real8 collinearTol, tdir[3];
75
76         collinearTol = 1.0 - cos(angleTol);
77         CrossVector(ndir[0], ndir[1], tdir);
78         NormalizeVec(tdir);
79
80         *planarJunc =
81             (fabs(fabs(DotProduct(tdir, tjunc)) - 1.0) <
82              collinearTol);
83     } else if (numArmEdge == 1) {
84         *planarJunc =
85             (fabs(DotProduct(ndir[0], tjunc)) < sin(angleTol));
86     } else {
87         *planarJunc = 1;
88     }
89 } else {
90     binaryJunc = -1;
91 }
92
93 return(binaryJunc);
94 }

```

Meaning. The classifier requires exactly one arm with $\|\mathbf{b}\| > 1 + 10^{-12}$ and two remaining non-junction arms. Each line direction is minimum-imaged and normalized. For the two non-junction arms it constructs $\mathbf{n} = \mathbf{b} \times \mathbf{t}$ and treats $\|\mathbf{n}\|^2 > 10^{-2}$ as edge-bearing. A five-degree tolerance then tests whether the junction direction follows the intersection of two parent planes, lies in the sole edge parent plane, or has two screw parents. Those cases are planar and are not split. The magnitude test calls the two remaining arms “glide” arms in the implementation; it does not independently validate a glissile Burgers-vector family.

4.1.3 Admitting the serial degree-three candidate

Code added or changed

```

1 int binaryJunc = -1;
2
3 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) &&
4     (node->numNbrs != 2)) {
5     REMOVE_CONSTRAINTS(node->constraint, CORNER_NODE);
6 }
7
8 if (node->numNbrs < 3) continue;
9
10 if (node->numNbrs == 3) {
11     int planarJunc;
12     real8 tjunc[3];
13
14     if ((!param->split3node) ||
15         (param->materialType != MAT_TYPE_BCC)) {
16         continue;
17     }
18
19     binaryJunc = BCC_binary_junction_node(home, node, tjunc,
20                                         &planarJunc);
21
22     if ((binaryJunc < 0) || planarJunc) continue;
23 }
24
25 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, SURFACE_NODE)) {
26     continue;
27 }
28
29 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, PINNED_NODE)) {
30     continue;
31 }

```

Meaning. The serial candidate floor changes from degree four to degree three, but the new degree-three path is narrow: it requires the switch, BCC material, a recognized non-planar binary junction, and an unpinned/non-surface node. The existing short-segment and cycle-frequency checks still apply later in the function.

4.1.4 Selecting only the two non-junction arms

Code changed

```

1 armList = (int *)calloc(1, (nbrs - 1) * sizeof(int));
2 armList2 = (int *)calloc(1, (nbrs - 1) * sizeof(int));
3

```

```

4 for (j = 0; j < numSets; j++) {
5     int tmpRepositionNode1, tmpRepositionNode2;
6
7     if ((nbrs == 3) && (armSets[j][binaryJunc] != 0)) {
8         continue;
9     }
10
11     /* Existing trial-split evaluation follows. */
12 }

```

Meaning. Two selected arms must fit in the work arrays, hence `nbrs - 1` instead of `nbrs - 2`. The pre-existing split-count table already contains three combinations for a degree-three node. The new filter discards the two combinations containing the junction arm, leaving only the trial that moves both non-junction arms.

4.1.5 Degree-two mobility boundary condition

Code added

```

1 AdjustJuncNodeForceAndVel(home, splitNode1, &mobArgs1);
2 AdjustJuncNodeForceAndVel(home, splitNode2, &mobArgs2);
3
4 if (nbrs == 3) {
5     Node_t *twoNode =
6         (splitNode1->numNbrs == 2) ? splitNode1 :
7         ((splitNode2->numNbrs == 2) ? splitNode2 :
8          (Node_t *)NULL);
9
10    if (twoNode != (Node_t *)NULL) {
11        twoNode->vX = 0.0;
12        twoNode->vY = 0.0;
13        twoNode->vZ = 0.0;
14    }
15 }

```

The same zeroing block is applied again after the repositioned trial has its mobility recomputed.

Meaning. Both trial nodes receive the established junction force/velocity adjustment. For this binary split, the resulting degree-two corner is stationary during the energetic test, matching the reference boundary condition and preventing it from contributing artificial power.

4.1.6 Guarding direction and accepting power

Code added or changed

```

1 if (((nbrs != 3) || !param->enforceGlidePlanes) &&
2     (vd1 > eps) && (vd2 > eps) &&
3     (fabs(vcl1Dotvcl2/vcl1Sqrt/vcl2Sqrt+1.0) < 0.01)) {
4     /* Existing equal-and-opposite velocity handling. */
5 }
6
7 if (((powerTest - powerMax) > eps) &&
8     ((nbrs != 3) || (powerTest > 1.0))) {
9     vel1[X] = splitNode1->vX;
10    vel1[Y] = splitNode1->vY;
11    vel1[Z] = splitNode1->vZ;
12 }

```

```

13     vel2[X] = splitNode2->vX;
14     vel2[Y] = splitNode2->vY;
15     vel2[Z] = splitNode2->vZ;
16
17     splitPos1[X] = splitNode1->x;
18     splitPos1[Y] = splitNode1->y;
19     splitPos1[Z] = splitNode1->z;
20
21     splitPos2[X] = splitNode2->x;
22     splitPos2[Y] = splitNode2->y;
23     splitPos2[Z] = splitNode2->z;
24 }

```

Meaning. The norm tests prevent division by zero when testing nearly opposite velocities. Degree-three handling also respects enforced glide planes. A binary-junction trial must improve on the current best power and exceed the absolute threshold `powerTest > 1.0`. Saving the winning velocities and trial positions already existed; the new degree-three behavior is the stricter acceptance condition and the later reuse of those saved positions. The serial path already used $d_{\text{split}} = 2r_{\text{ann}} + 10^{-12}$ before this feature; that line is supporting context, not a newly introduced serial change.

4.1.7 Using the evaluated position and preserving the corner

Code added

```

1  if (nbrs == 3) {
2      VECTOR_COPY(pos1, splitPos1);
3      VECTOR_COPY(pos2, splitPos2);
4  }
5
6  /* SplitNode() executes the accepted final split here. */
7
8  if (nbrs == 3) {
9      if (splitNode1->numNbrs == 2) {
10         ADD_CONSTRAINTS(splitNode1->constraint, CORNER_NODE);
11         if (globalOp) {
12             AddOpResetConstraint(home, &splitNode1->myTag,
13                                 splitNode1->constraint);
14         }
15     }
16     if (splitNode2->numNbrs == 2) {
17         ADD_CONSTRAINTS(splitNode2->constraint, CORNER_NODE);
18         if (globalOp) {
19             AddOpResetConstraint(home, &splitNode2->myTag,
20                                 splitNode2->constraint);
21         }
22     }
23 }

```

Meaning. The final degree-three split reuses the positions that actually passed the trial mobility/power test instead of entering the general `CalcJunctionLen()` path. After `SplitNode()` succeeds, whichever output node has degree two receives `CORNER_NODE`. A global operation then appends the full constraint reset after the create/connectivity records.

5 Parallel candidate and force evaluation

5.1 src/SMNSupport.cc

Code added: shared parallel eligibility predicate

```
1 static int IsSplittableMultiNode(Home_t *home, Node_t *node)
2 {
3     int    binaryJunc, planarJunc;
4     real8 tjunc[3];
5
6     if (node->numNbrs < 3) {
7         return(0);
8     }
9
10    if (node->numNbrs > 3) {
11        return(1);
12    }
13
14    if (HAS_ANY_OF_CONSTRAINTS(node->constraint,
15                               SURFACE_NODE | PINNED_NODE)) {
16        return(0);
17    }
18
19    if ((!home->param->split3node) ||
20        (home->param->materialType != MAT_TYPE_BCC)) {
21        return(0);
22    }
23
24    binaryJunc = BCC_binary_junction_node(home, node, tjunc,
25                                           &planarJunc);
26
27    return((binaryJunc >= 0) && !planarJunc);
28 }
```

Code changed: degree-three enumeration and lists

```
1 maxSets = POSSIBLE_SPLITS[numNbrs];
2 maxSplitCnt = numNbrs >> 1;
3
4 if (numNbrs == 3) {
5     maxSplitCnt = 2;
6 }
7
8 /* BuildRecvDomList(): */
9 if (lmnInfo[i].multiNode->numNbrs < 3) {
10     continue;
11 }
12
13 /* Applied in both local and ghost list builders: */
14 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) &&
15     (node->numNbrs != 2)) {
16     REMOVE_CONSTRAINTS(node->constraint, CORNER_NODE);
17 }
18
19 if (!IsSplittableMultiNode(home, node)) {
20     continue;
21 }
```

Meaning. Native and ghost lists now agree on the exact degree-three eligibility rules. `SMNGetArmSets()` is allowed to select two arms from a three-arm node; the existing `POSSIBLE_SPLITS[3] == 3` table entry supplies the three combinations. Receive-domain construction also admits degree three, and both list builders clear stale corner state before applying the predicate. The unchanged `src/SplitMultiNodesParallel.cc` then orchestrates these lists, partial-force communication, and final owning-domain evaluation.

5.2 `src/SMNEvalSplitForces.cc`

This stage evaluates local force contributions for native and ghost candidates before point-to-point force exchange and accumulation on the owning domain.

Code changed: distance, classification, and arm selection

```

1  eps = 1.0e-12;
2  splitDist = param->rann * 2.0 + eps;
3
4  int binaryJunc = -1;
5
6  if (numNbrs == 3) {
7      int planarJunc;
8      real8 tjunc[3];
9
10     binaryJunc = BCC_binary_junction_node(home, node, tjunc,
11                                           &planarJunc);
12     if (planarJunc) binaryJunc = -1;
13 }
14
15 armList = (int *)calloc(1, (numNbrs - 1) * sizeof(int));
16
17 for (setID = 0; setID < totalSets; setID++) {
18     int tmpRepositionNode1 = 0, tmpRepositionNode2 = 0;
19
20     if ((numNbrs == 3) &&
21         ((binaryJunc < 0) ||
22          (armSets[setID][binaryJunc] != 0))) {
23         continue;
24     }
25
26     /* Existing partial-force trial follows. */
27 }

```

Code changed: mobility status and degree-two velocity

```

1  mobStatus = EvaluateMobility(home, splitNode1, &mobArgs1);
2  mobStatus |= EvaluateMobility(home, splitNode2, &mobArgs2);
3
4  AdjustJuncNodeForceAndVel(home, splitNode1, &mobArgs1);
5  AdjustJuncNodeForceAndVel(home, splitNode2, &mobArgs2);
6
7  if (numNbrs == 3) {
8      Node_t *twoNode =
9          (splitNode1->numNbrs == 2) ? splitNode1 :
10          ((splitNode2->numNbrs == 2) ? splitNode2 :
11           (Node_t *)NULL);
12
13     if (twoNode != (Node_t *)NULL) {

```

```

14     twoNode->vX = 0.0;
15     twoNode->vY = 0.0;
16     twoNode->vZ = 0.0;
17 }
18 }
19
20 if (((numNbrs != 3) || !param->enforceGlidePlanes) &&
21     (vd1 > eps) && (vd2 > eps) &&
22     (fabs(vellDotvel2/vd1Sqrt/vd2Sqrt+1.0) < 0.01)) {
23     /* Existing equal-and-opposite velocity handling. */
24 }

```

Meaning. The positive epsilon places the trial connector just beyond the immediate corner-remesh threshold. Reclassification protects against stale list data, and only the arm set excluding the junction arm is evaluated. Work storage fits two selected arms and reposition flags start deterministically at zero. Using `!=` retains an error from either mobility call. Both trial nodes are adjusted, the degree-two node is stationary, and velocity normalization cannot divide by a zero norm.

5.3 src/SMNEvalMultiNodeSplits.cc

This owning-domain stage receives the complete forces, repeats the mobility and power test, and executes the accepted topology change.

Code changed: candidate parity with the force stage

```

1  eps      = 1.0e-12;
2  vNoise   = param->splitMultiNodeAlpha * param->rTol / param->deltaTT;
3  splitDist = param->rann * 2.0 + eps;
4
5  int binaryJunc = -1;
6
7  if (numNbrs == 3) {
8      int planarJunc;
9      real8 tjunc[3];
10
11     binaryJunc = BCC_binary_junction_node(home, node, tjunc,
12                                           &planarJunc);
13     if (planarJunc) binaryJunc = -1;
14 }
15
16 armList = (int *)calloc(1, (numNbrs - 1) * sizeof(int));
17 armList2 = (int *)calloc(1, (numNbrs - 1) * sizeof(int));
18
19 for (setID = 0; setID < totalSets; setID++) {
20     int tmpRepositionNode1 = 0, tmpRepositionNode2 = 0;
21
22     if ((numNbrs == 3) &&
23         ((binaryJunc < 0) ||
24          (armSets[setID][binaryJunc] != 0))) {
25         continue;
26     }
27
28     /* Existing final trial evaluation follows. */
29 }

```

Code added: degree-two boundary condition

```
1 AdjustJuncNodeForceAndVel(home, splitNode1, &mobArgs1);
2 AdjustJuncNodeForceAndVel(home, splitNode2, &mobArgs2);
3
4 if (numNbrs == 3) {
5     Node_t *twoNode =
6         (splitNode1->numNbrs == 2) ? splitNode1 :
7         ((splitNode2->numNbrs == 2) ? splitNode2 :
8         (Node_t *)NULL);
9
10    if (twoNode != (Node_t *)NULL) {
11        twoNode->vX = 0.0;
12        twoNode->vY = 0.0;
13        twoNode->vZ = 0.0;
14    }
15 }
```

This block occurs after both the coincident trial mobility and the repositioned trial mobility.

Code changed: direction guard, power, final positions, and constraint

```
1 if (((numNbrs != 3) || !param->enforceGlidePlanes) &&
2     (vd1 > eps) && (vd2 > eps) &&
3     (fabs(vellDotvel2/vd1Sqrt/vd2Sqrt+1.0) < 0.01)) {
4     /* Existing equal-and-opposite velocity handling. */
5 }
6
7 if (((powerTest - powerMax) > eps) &&
8     ((numNbrs != 3) || (powerTest > 1.0))) {
9     /* Save winning velocities and splitPos1/splitPos2. */
10 }
11
12 if (numNbrs == 3) {
13     VECTOR_COPY(pos1, splitPos1);
14     VECTOR_COPY(pos2, splitPos2);
15 }
16
17 /* SplitNode() executes the accepted final split here. */
18
19 if (numNbrs == 3) {
20     if (splitNode1->numNbrs == 2) {
21         ADD_CONSTRAINTS(splitNode1->constraint, CORNER_NODE);
22         if (globalOp) {
23             AddOpResetConstraint(home, &splitNode1->myTag,
24                                 splitNode1->constraint);
25         }
26     }
27     if (splitNode2->numNbrs == 2) {
28         ADD_CONSTRAINTS(splitNode2->constraint, CORNER_NODE);
29         if (globalOp) {
30             AddOpResetConstraint(home, &splitNode2->myTag,
31                                 splitNode2->constraint);
32         }
33     }
34 }
```

Meaning. The owning-domain decision mirrors the serial decision: the degree-two trial is stationary, power must both improve and exceed one, and the final split uses the tested positions. As in the serial function, storing the winning positions was pre-existing; the new code applies the absolute degree-three threshold and explicitly copies those positions into the final degree-three split. Successful corner marking is immediately followed by a remote constraint operation, so serial and parallel topology selectors produce the same persistent topology state.

6 Remesh preservation and MPI propagation

6.1 src/Remesh.cc

Both remesh rules reach the shared `MeshCoarsen()` implementation, so the corner rule is implemented once in its candidate and execution phases.

Code added: candidate-phase guard

```

1 if ((HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
2     HAS_ANY_OF_CONSTRAINTS(nbr1->constraint, CORNER_NODE)) &&
3     (r1 > (2.0 * param->rann)) &&
4     (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
5     HAS_ANY_OF_CONSTRAINTS(nbr2->constraint, CORNER_NODE)) &&
6     (r2 > (2.0 * param->rann))) {
7     continue;
8 }
```

Code added: independent execution-phase gates

```

1 mergeNbr1Allowed = 1;
2 mergeNbr2Allowed = 1;
3
4 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
5     HAS_ANY_OF_CONSTRAINTS(nbr1->constraint, CORNER_NODE)) {
6     real8 dx1, dy1, dz1;
7     real8 cornerMergeDist2;
8
9     dx1 = nbr1->x - node->x;
10    dy1 = nbr1->y - node->y;
11    dz1 = nbr1->z - node->z;
12
13    ZImage(param, &dx1, &dy1, &dz1);
14    cornerMergeDist2 = 4.0 * param->rann * param->rann;
15
16    mergeNbr1Allowed = ((dx1*dx1 + dy1*dy1 + dz1*dz1) <=
17                        cornerMergeDist2);
18 }
19
20 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
21     HAS_ANY_OF_CONSTRAINTS(nbr2->constraint, CORNER_NODE)) {
22     real8 dx2, dy2, dz2;
23     real8 cornerMergeDist2;
24
25     dx2 = nbr2->x - node->x;
26     dy2 = nbr2->y - node->y;
27     dz2 = nbr2->z - node->z;
28
29     ZImage(param, &dx2, &dy2, &dz2);
```

```

30     cornerMergeDist2 = 4.0 * param->rann * param->rann;
31
32     mergeNbr2Allowed = ((dx2*dx2 + dy2*dy2 + dz2*dz2) <=
33                         cornerMergeDist2);
34 }
35
36 if ((!mergeNbr1Allowed) && (!mergeNbr2Allowed)) {
37     continue;
38 }
39
40 if (mergeNbr1Allowed &&
41     ((nbr1->flags & NO_MESH_COARSEN) == 0)) {
42     /* Existing MergeNode() attempt for nbr1. */
43 }
44
45 if (mergeDone == 0) {
46     if (mergeNbr2Allowed &&
47         ((nbr2->flags & NO_MESH_COARSEN) == 0)) {
48         /* Existing MergeNode() attempt for nbr2. */
49     }
50 }

```

Meaning. An edge touching a corner may be merged only if its minimum-image length is $L \leq 2r_{\text{ann}}$. The candidate guard skips a degree-two coarsen candidate when both relevant corner edges are long, but retains an escape for pathologically short segments. At execution, each neighbor is gated independently. This prevents arbitrary arm ordering from collapsing a long protected edge merely because the other edge is short.

6.2 src/RemoteOps.cc

Code added

```

1 void AddOpResetConstraint(Home_t *home, Tag_t *tag1, int constraint)
2 {
3     #ifdef _OPENMP
4     #pragma omp critical (CRIT_ADD_OP)
5     #endif
6     {
7         int                opDataLen = sizeof(RemOpResetConstraint_t);
8         int                neededLen = home->opListUsedLen + opDataLen;
9         RemOpResetConstraint_t opData;
10
11         if (home->opListAllocLen < neededLen) {
12             ExtendOpList(home);
13         }
14
15         opData.opType = REM_OP_RESET_CONSTRAINT;
16         opData.tag1.domainID = tag1->domainID;
17         opData.tag1.index    = tag1->index;
18         opData.constraint    = constraint;
19
20         memcpy(&home->opListBuf[home->opListUsedLen], &opData, opDataLen);
21         home->opListUsedLen += opDataLen;
22     }
23
24     return;
25 }

```

Meaning. The emitter grows the operation buffer when necessary, packs the complete constraint value, and appends the fixed-size record. The same OpenMP critical section used by other operation emitters prevents concurrent writers from overlapping records.

6.3 src/FixRemesh.cc

Code added

```

1 } else if (opType == REM_OP_RESET_CONSTRAINT) {
2     int opLen = sizeof(RemOpResetConstraint_t);
3     RemOpResetConstraint_t opBuf;
4
5     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
6     offSet += opLen;
7
8     node1 = GetNodeFromTag(home, opBuf.tag1);
9
10    if (node1 != (Node_t *)NULL) {
11        SET_CONSTRAINTS(node1->constraint, opBuf.constraint);
12    }

```

Meaning. The receiver consumes exactly one matching wire record, resolves the target tag, and replaces the complete constraint mask. Ordering is essential: `SplitNode()` emits create/connectivity operations first, then the caller appends the reset. Replay therefore creates the remote ghost before setting `CORNER_NODE`, and it does so before immediate remeshing.

7 Regression fixture

7.1 tests/bcc_binary_junction_node.ctrl

The control fixture is shown in full because it is both executable input and the run guide.

File added

```

1 #
2 # One-step regression fixture for splitting a non-planar BCC binary-junction
3 # node. Run from the tests directory with:
4 #
5 #   ../bin/paradis -d bcc_binary_junction_node.data \
6 #       bcc_binary_junction_node.ctrl
7 #
8 # The final restart must contain five nodes. The original three-arm node is
9 # split into a three-arm node and a degree-two node carrying constraint 16
10 # (CORNER_NODE). Set useParallelSplitMultiNode to 1 to exercise the parallel
11 # split implementation; 0 exercises the serial implementation.
12 # A two-rank communication check can then be run with:
13 #
14 #   mpirun -n 2 ../bin/paradis -doms 2 1 1 \
15 #       -d bcc_binary_junction_node.data bcc_binary_junction_node.ctrl
16 #
17 dirname = "bcc_binary_junction_node_results"
18
19 numXdoms = 1
20 numYdoms = 1
21 numZdoms = 1
22

```

```

23 numXcells = 4
24 numYcells = 4
25 numZcells = 4
26
27 fmEnabled      = 1
28 fmMPOrder      = 2
29 fmTaylorOrder  = 5
30 fmCorrectionTbl = "../inputs/fm-ctab.Ta.600K.0GPa.m2.t5.dat"
31
32 maxstep = 1
33 remeshRule = 2
34 minSeg = 2.000000e+02
35 maxSeg = 2.500000e+03
36 rann = 2.000000e+00
37 rTol = 1.000000e+00
38 timestepIntegrator = "trapezoid"
39
40 rc = 5.000000e+00
41 MobScrew = 1.000000e+01
42 MobEdge = 1.000000e+01
43 MobClimb = 1.000000e-08
44 mobilityLaw = "BCC_0"
45
46 split3node = 1
47 splitMultiNodeFreq = 1
48 useParallelSplitMultiNode = 0
49
50 loadType = 0
51 edotdir = [
52   0.0
53   0.0
54   1.0
55 ]
56 appliedStress = [
57   0.000000e+00
58   0.000000e+00
59   5.000000e+08
60   0.000000e+00
61   0.000000e+00
62   0.000000e+00
63 ]
64
65 savetimers = 0
66 savecn = 1
67 savecnfreq = 1
68 savecncounter = 0
69 gnuplot = 0
70 povray = 0
71 saveprop = 0

```

Meaning. The fixture runs exactly one BCC step with a zz stress of 5×10^8 , evaluates multi-node splitting every cycle, and writes a restart. It defaults to `useParallelSplitMultiNode = 0`, the serial topology implementation. Set the selector to one in a `PARALLEL` build to exercise the distributed force path. The two-rank command overrides the domain layout to $2 \times 1 \times 1$, testing ownership and remote constraint replay.

7.2 tests/bcc_binary_junction_node.data

```

File added
1 dataFileVersion = 5
2 numFileSegments = 1
3 minCoordinates = [
4   -1.750000e+04
5   -1.750000e+04
6   -1.750000e+04
7 ]
8 maxCoordinates = [
9   1.750000e+04
10  1.750000e+04
11  1.750000e+04
12 ]
13 nodeCount = 4
14 dataDecompType = 2
15 dataDecompGeometry = [
16   1
17   1
18   1
19 ]
20
21 domainDecomposition =
22 # Dom_ID  Minimum XYZ bounds          Maximum XYZ bounds
23   0        -17500.0 -17500.0 -17500.0    17500.0 17500.0 17500.0
24
25 nodalData =
26 # A non-planar BCC binary junction: the <100> junction arm balances the two
27 # 1/2<111> glide arms. The three fixed end nodes isolate the topology
28 # operation from endpoint motion.
29 0,0 0.0 0.0 0.0 0.0 3 0
30   0,1 -1.1547005383792517  0.0          0.0
31        0.0                1.0          0.0
32   0,2  0.5773502691896258  0.5773502691896258  0.5773502691896258
33        -0.7071067811865475  0.0          0.7071067811865475
34   0,3  0.5773502691896258 -0.5773502691896258 -0.5773502691896258
35        -0.7071067811865475 -0.7071067811865475  0.0
36
37 0,1 2000.0 0.0 0.0 0.0 1 7
38   0,0  1.1547005383792517  0.0          0.0
39        0.0                1.0          0.0
40
41 0,2 0.0 2000.0 0.0 0.0 1 7
42   0,0 -0.5773502691896258 -0.5773502691896258 -0.5773502691896258
43        -0.7071067811865475  0.0          0.7071067811865475
44
45 0,3 0.0 0.0 2000.0 0.0 1 7
46   0,0 -0.5773502691896258  0.5773502691896258  0.5773502691896258
47        -0.7071067811865475 -0.7071067811865475  0.0

```

Meaning. Node 0,0 is the unconstrained degree-three candidate at the origin. The arm toward node 0,1 has a $\langle 100 \rangle$ Burgers-vector magnitude of approximately 1.1547 and is recognized as the junction arm. The two unit-magnitude $\frac{1}{2}\langle 111 \rangle$ arms balance it and form a non-planar geometry under the five-degree test. Nodes 0,1, 0,2, and 0,3 are 2000 units away and carry constraint 7 (fully pinned), isolating the topology event from endpoint motion.

7.3 Commands and expected result

Run from `tests`:

```
1 ../bin/paradis -d bcc_binary_junction_node.data \  
2   bcc_binary_junction_node.ctrl
```

For the parallel topology selector, change `useParallelSplitMultiNode = 1`. For a two-rank ownership test, run:

```
1 mpirun -n 2 ../bin/paradis -doms 2 1 1 \  
2   -d bcc_binary_junction_node.data \  
3   bcc_binary_junction_node.ctrl
```

After one step, `bcc_binary_junction_node_results/restart/restart.data` must contain five nodes:

- the three original pinned degree-one endpoints;
- exactly one degree-two node with constraint 16 (`CORNER_NODE`); and
- one degree-three node without the corner constraint.

The regression review completed serial and `PARALLEL` builds, obtained byte-identical one-rank restart data from selectors zero and one, completed two-rank MPI runs with both selectors, and observed the expected unique degree-two constraint-16 node.

8 End-to-end meaning of the change

The code fragments above form one ordered topology transaction:

1. Parameter plumbing enables degree-three BCC candidates.
2. The shared classifier identifies exactly one large-magnitude junction arm and rejects planar parent geometry.
3. Serial or parallel arm enumeration moves only the two non-junction arms in the trial split.
4. Mobility evaluation holds the degree-two trial node fixed, checks the separation direction, and accepts only a sufficiently positive power gain.
5. The final split reuses the tested positions and marks the degree-two result as a corner.
6. The append-only remote operation copies the full constraint state to ghost nodes before remeshing.
7. Corner-aware coarsening preserves long physical edges while retaining a controlled escape for very short pathological segments.

The change deliberately does not generalize all degree-three splitting. It is limited to BCC, exactly three valid arms, exactly one junction arm under the Burgers-magnitude rule, non-planar geometry, and nodes that are neither pinned nor surface-constrained. Nodes with four or more arms retain their established multi-node split behavior.